

Deep Learning

Lecture 2

Deep Learning Basics

- ImageClassification:



This image by Nikita is
licensed under [CC-BY2.0](#)

→ cat

Challenges

- Viewpoint variation
- Scale variation
- Deformation
- Occlusion
- Illumination conditions
- Background clutter
- Intra-class variation

Viewpoint variation



Illumination conditions



Scale variation



Deformation



Occlusion



Background clutter

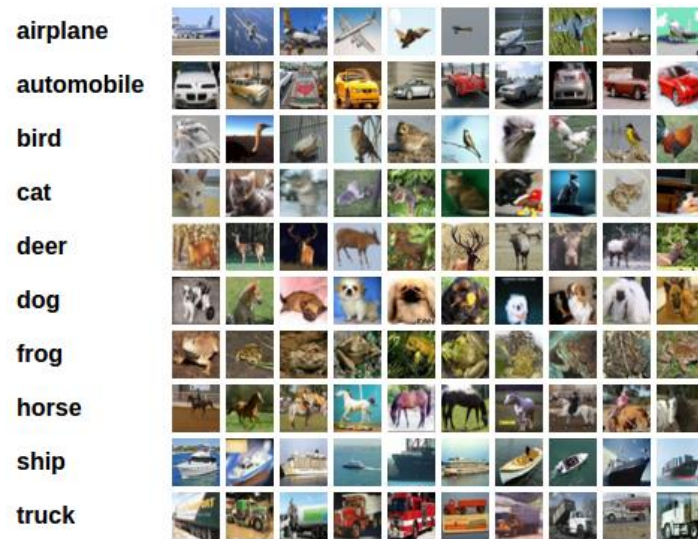


Intra-class variation



Solution: data-driven approach (CIFAR-10)

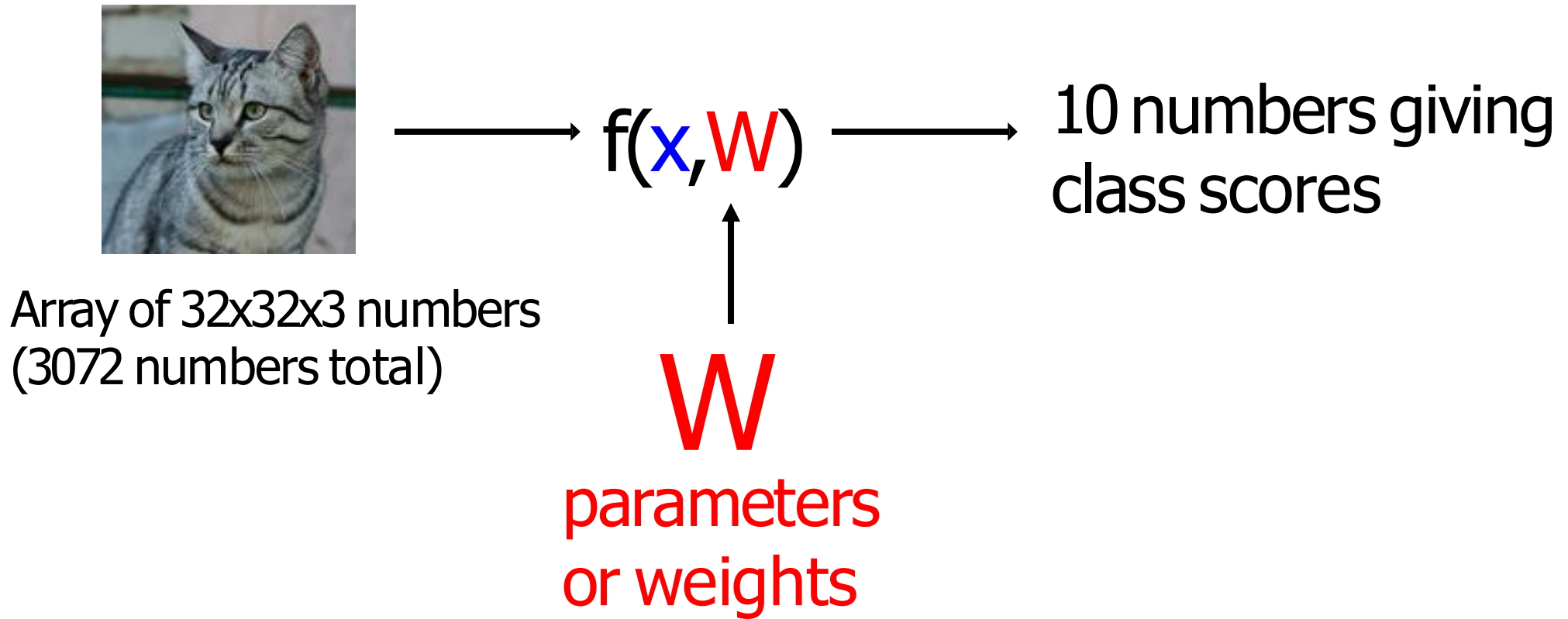
- 10 classes
- 50,000 training images
- 10,000 testing images



Test images and nearest neighbors



Linear Classifier



Linear Classifier

- We optimize the weights W

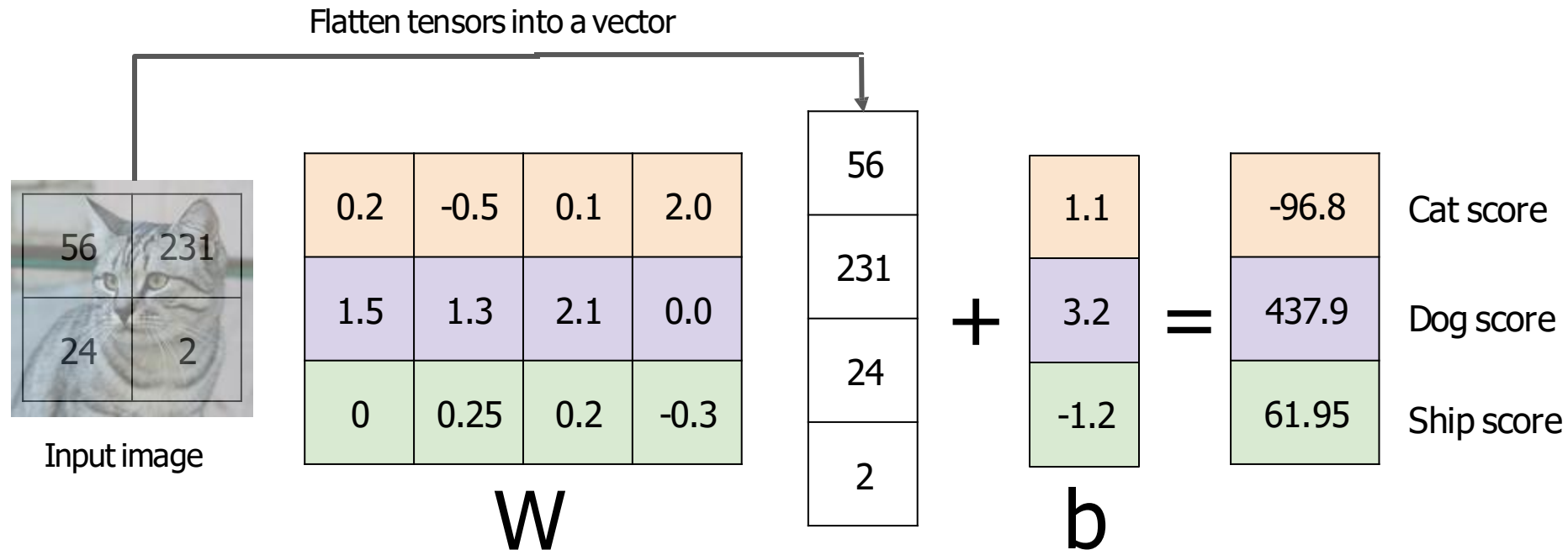


→ 10 classes

$$\boxed{f(x, W)}_{10 \times 1} = \overset{10 \times 3072}{\boxed{W}}_{3072 \times 1} \boxed{x}_{3072 \times 1} + \boxed{b}_{10 \times 1}$$

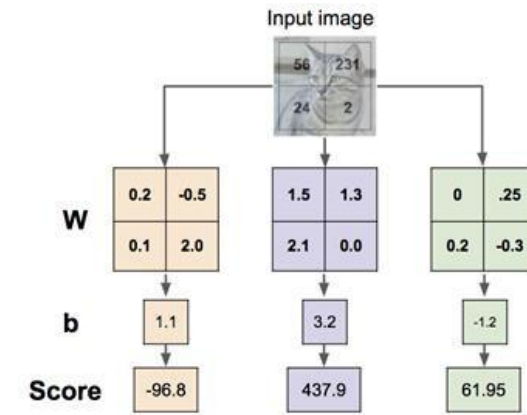
Algebraic View:

4 pixels, and 3 classes (cat/dog/ship)



Another view, a Template Matcher

- Each class has dedicated weights
- The score is based on pattern matching

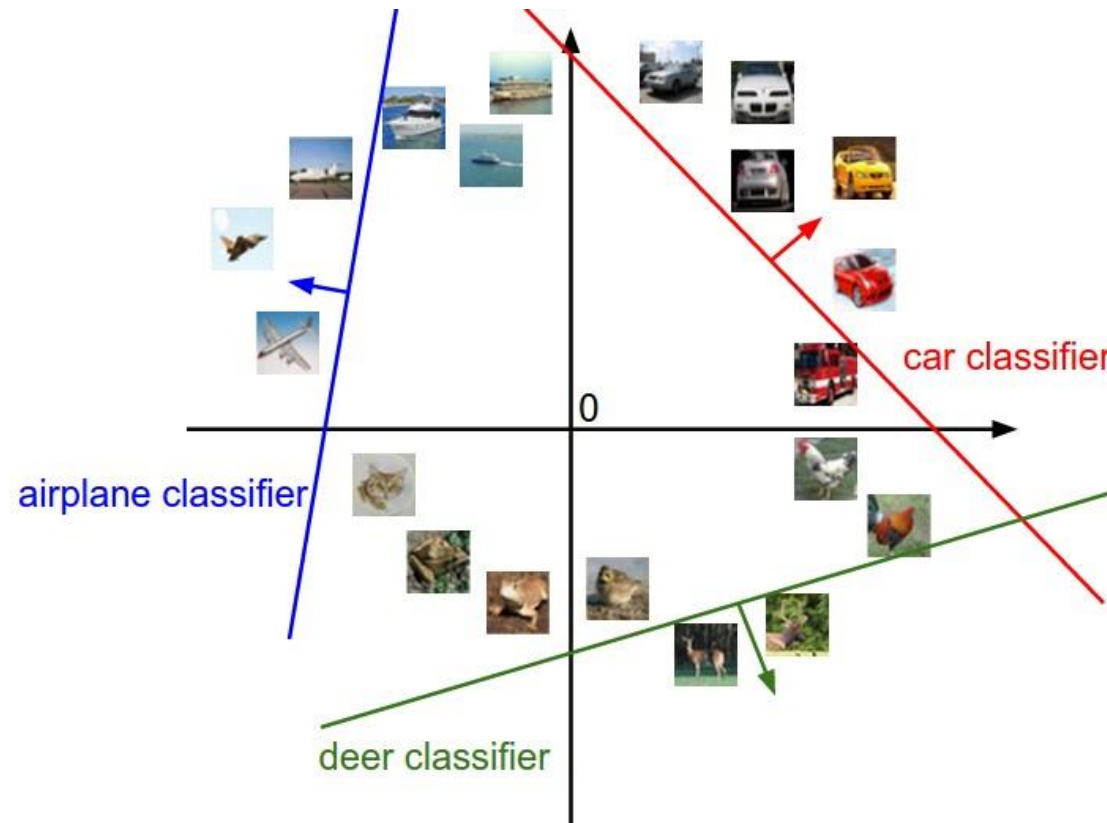


- Visualizing trained weights:



Geometric View:

Each neuron defines a linear boundary



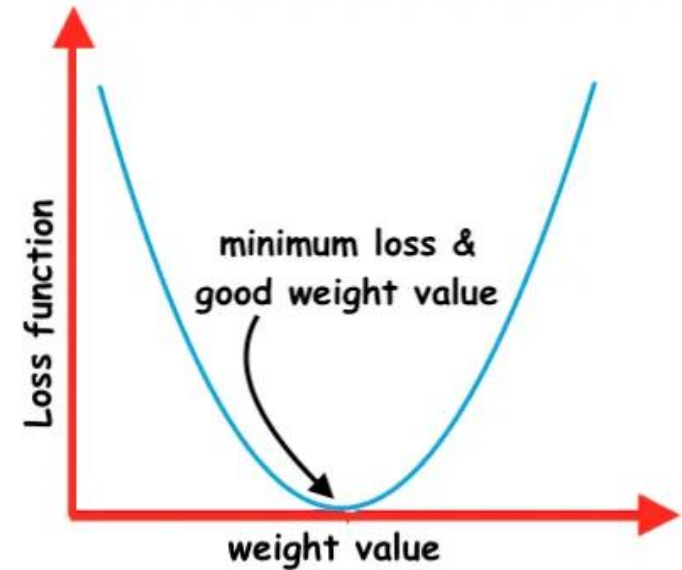
Loss Function

- Loss function or cost function assess model f is on training data $\{x_i, y_i\}_{i=1}^N$
- Requirements:

- It can be computed as sample average, i.e.,

$$L = \sum \frac{1}{n} L_i(x_i)$$

- High value when predictions are wrong
 - Low value when predictions are correct



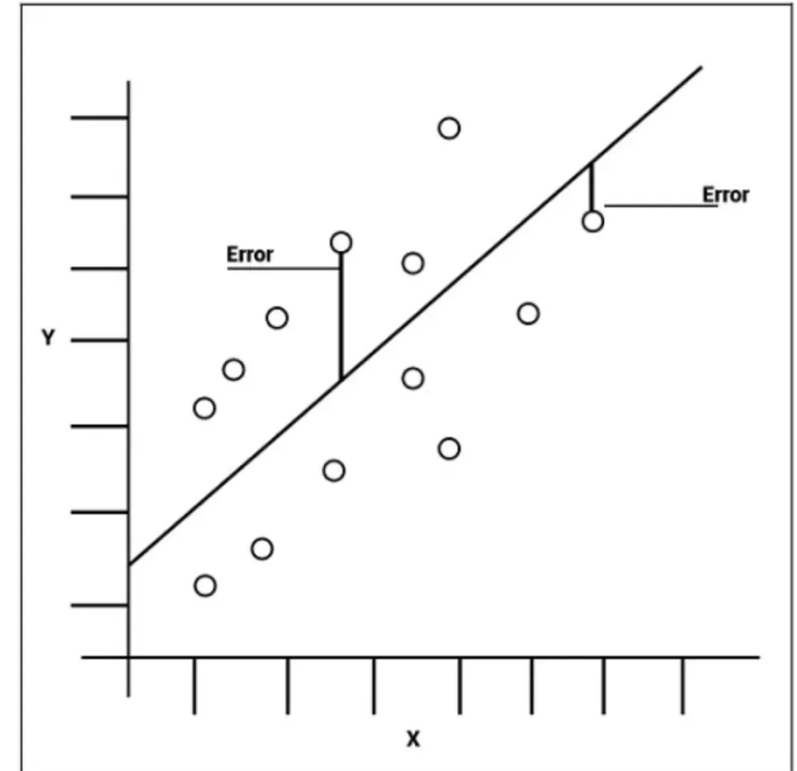
E.g., Mean Square Error (MSE)

- Regression, Mean Square Error (MSE)

$$L_i = \frac{1}{2} (y_i - f(x_i))^2$$

- Thus,

$$L = \frac{1}{n} \sum_i L_i$$



What about classification?

3 training sample;

$$f(x, W) = Wx$$

3 classes

x_1



x_2



x_3



cat

3.2

1.3

2.2

car

5.1

4.9

2.5

frog

-1.7

2.0

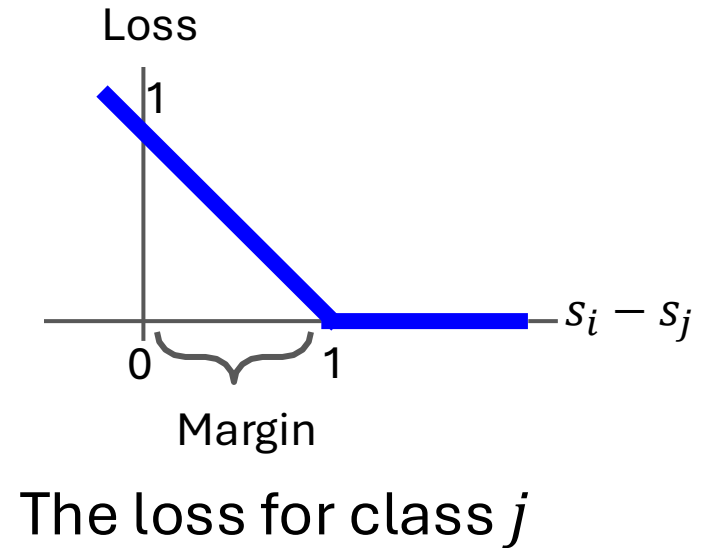
-3.1

What would be a good loss for classification?

Margin Loss (“Hinge Loss”)

- The score of the correct class (s_i) should be higher than all other class scores (s_j)

$$L_i = \sum_{j \neq i} \max(0, \underset{\substack{\uparrow \\ \text{Margin}}}{1 - (s_i - s_j)})$$



Margin Loss

$$= \max(0, 5.1 - 3.2 + 1) + \max(0, -1.7 - 3.2 + 1)$$

$$= \max(0, 2.9) + \max(0, -3.9)$$

$$= 2.9 + 0$$

$$= 2.9$$

$$L_i = \sum_{j \neq i} \max(0, 1 - (s_i - s_j))$$



cat

3.2

1.3

2.2

car

5.1

4.9

2.5

frog

-1.7

2.0

-3.1

Losses:

2.9

Margin Loss

$$= \max(0, 1.3 - 4.9 + 1) + \max(0, 2.0 - 4.9 + 1)$$

$$= \max(0, -2.6) + \max(0, -1.9)$$

$$= 0 + 0 = 0$$

$$L_i = \sum_{j \neq i} \max(0, 1 - (s_i - s_j))$$



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	

Margin Loss

$$L_i = \sum_{j \neq i} \max(0, 1 - (s_i - s_j))$$

$$= \max(0, 2.2 - (-3.1) + 1) + \max(0, 2.5 - (-3.1) + 1)$$

$$= \max(0, 6.3) + \max(0, 6.6)$$

$$= 6.3 + 6.6$$

$$= 12.9$$



cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

Margin Loss

$$L = (2.9 + 0 + 12.9) / 3 = 5.27$$

$$L_i = \sum_{j \neq i} \max(0, 1 - (s_i - s_j))$$



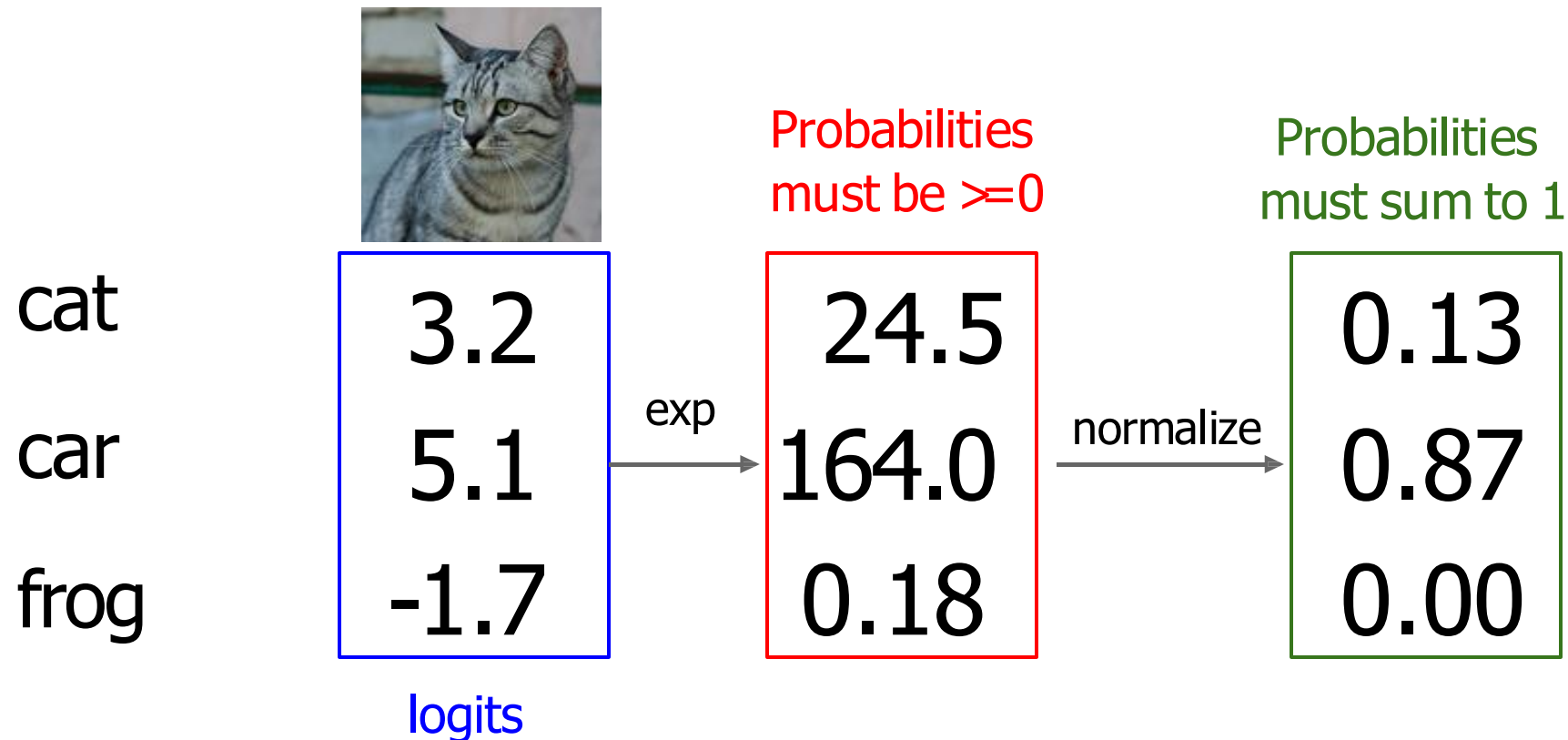
cat	3.2	1.3	2.2
car	5.1	4.9	2.5
frog	-1.7	2.0	-3.1
Losses:	2.9	0	12.9

Margin Loss Properties

- Margin loss only cares that the correct class is ahead by a margin not ***how much***.
- This can make the scores not well calibrated.
- When to use?
 - Low data
 - You care more about classification than “true” scores
 - You want robustness to outliers

Cross-entropy Loss

- Align the prediction with the training data distribution (maximum likelihood); first, turn the labels to probabilities (Softmax):



Cross-entropy

- The *cross-entropy* between the “true” distribution p , and estimated distribution q is defined as:

$$\text{Cross-Entropy}(p, q) = H(p) + KL(p, q)$$

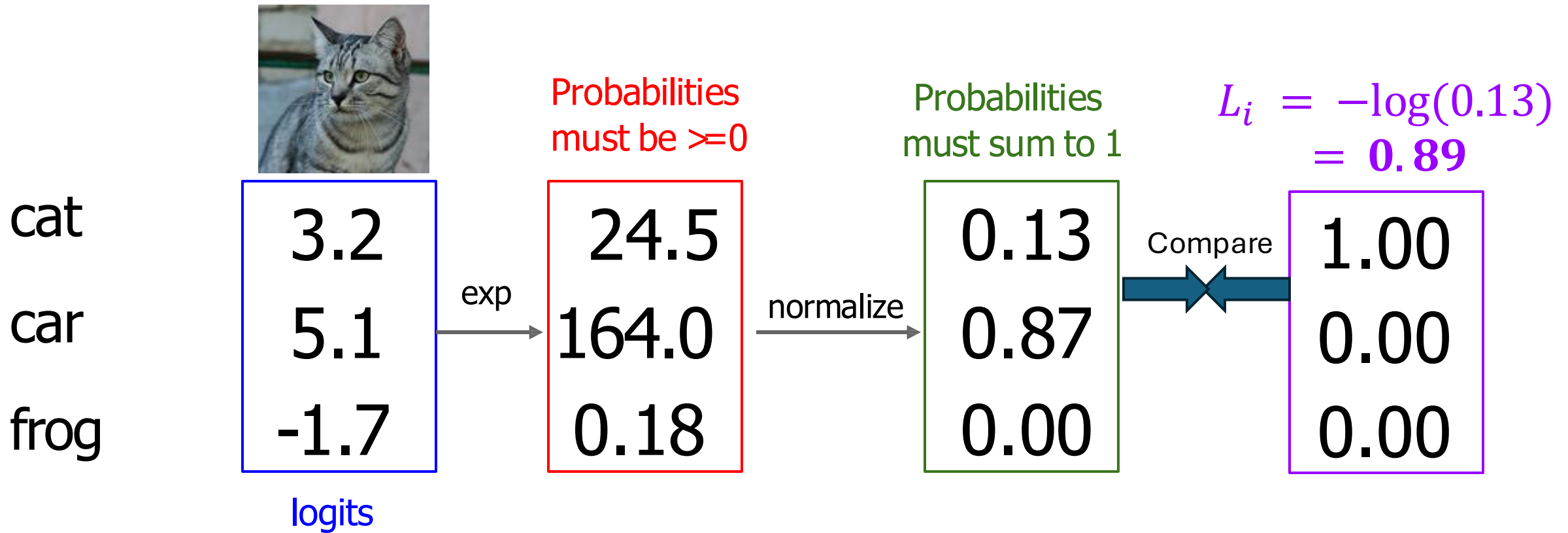
$$= 0 - \sum_x p(x) \log(q(x))$$


“true” distribution

$$\text{i.e., } p = [0, \dots, \underbrace{1}, \dots, 0]$$

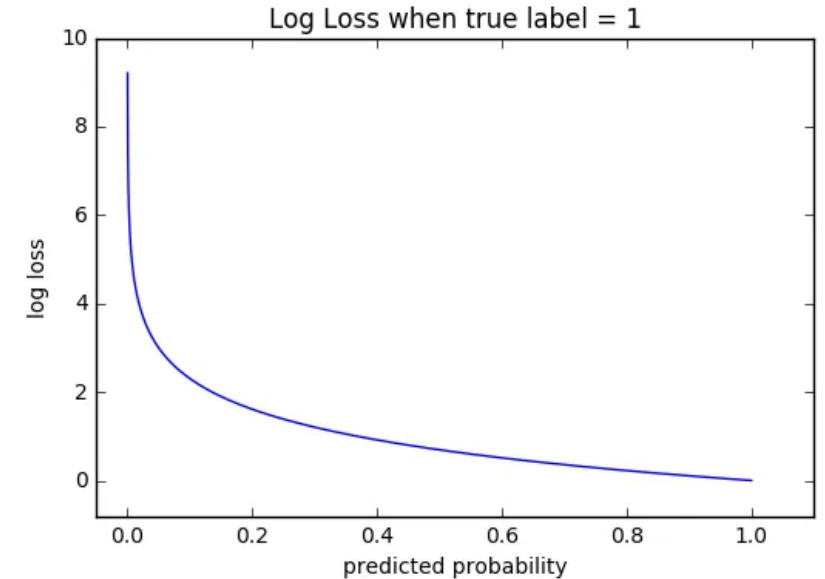
True class

Calculating the Loss



Cross-entropy properties

- **Smooth, non-saturating loss** that keep learning active
 - Never zero
- Better calibration and interpretability



Up until now

- We've shown two types of loss:
 - Margin loss – focuses only on the boundary between classes
 - Cross-entropy – matches model predictions to the training distribution
- There are others*
- Losses can also rely on external models

* - <https://docs.pytorch.org/docs/stable/nn.html#loss-functions>

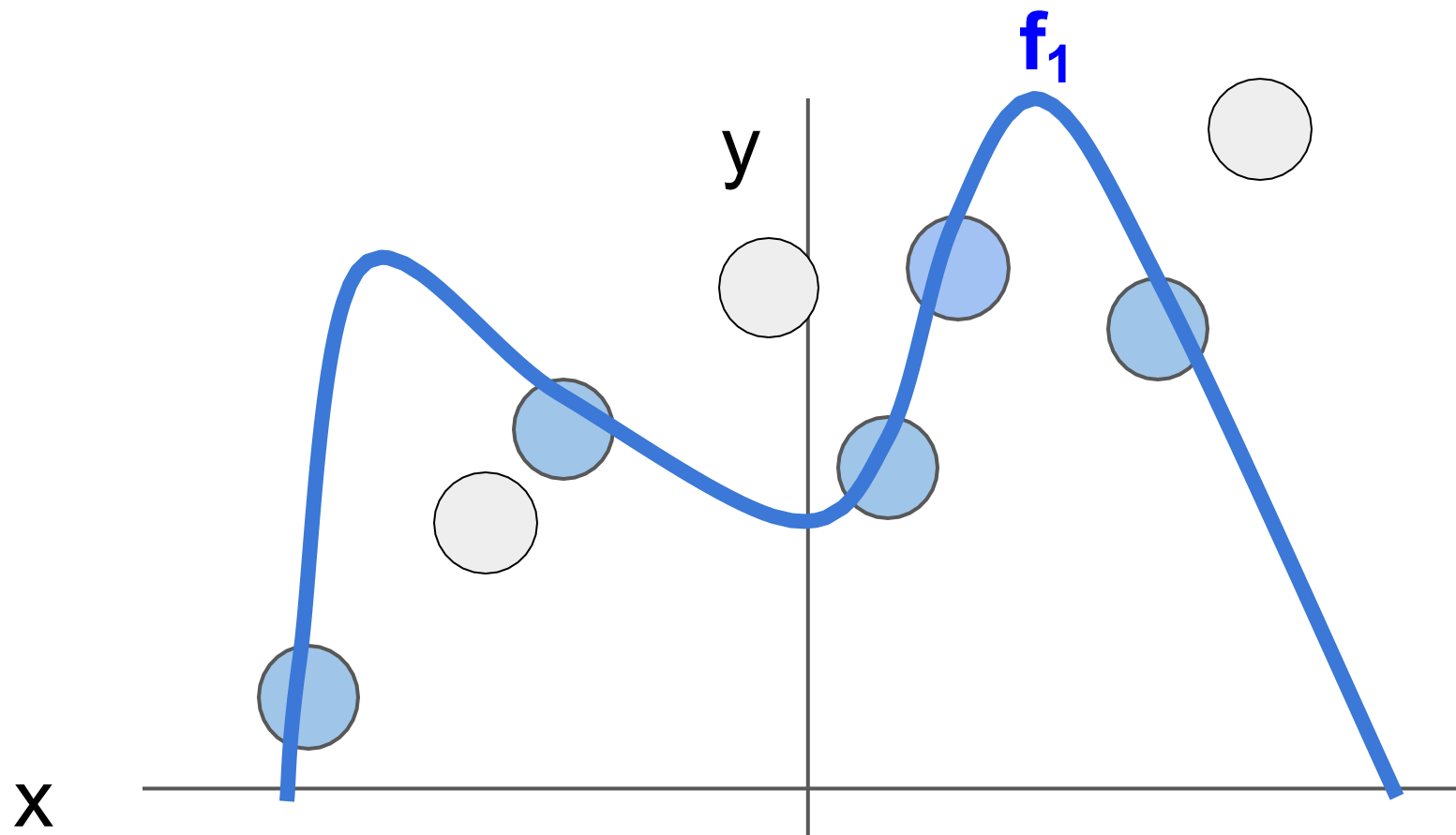
Regularization

$$L(W) = \underbrace{\frac{1}{N} \sum_{i=1}^N L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

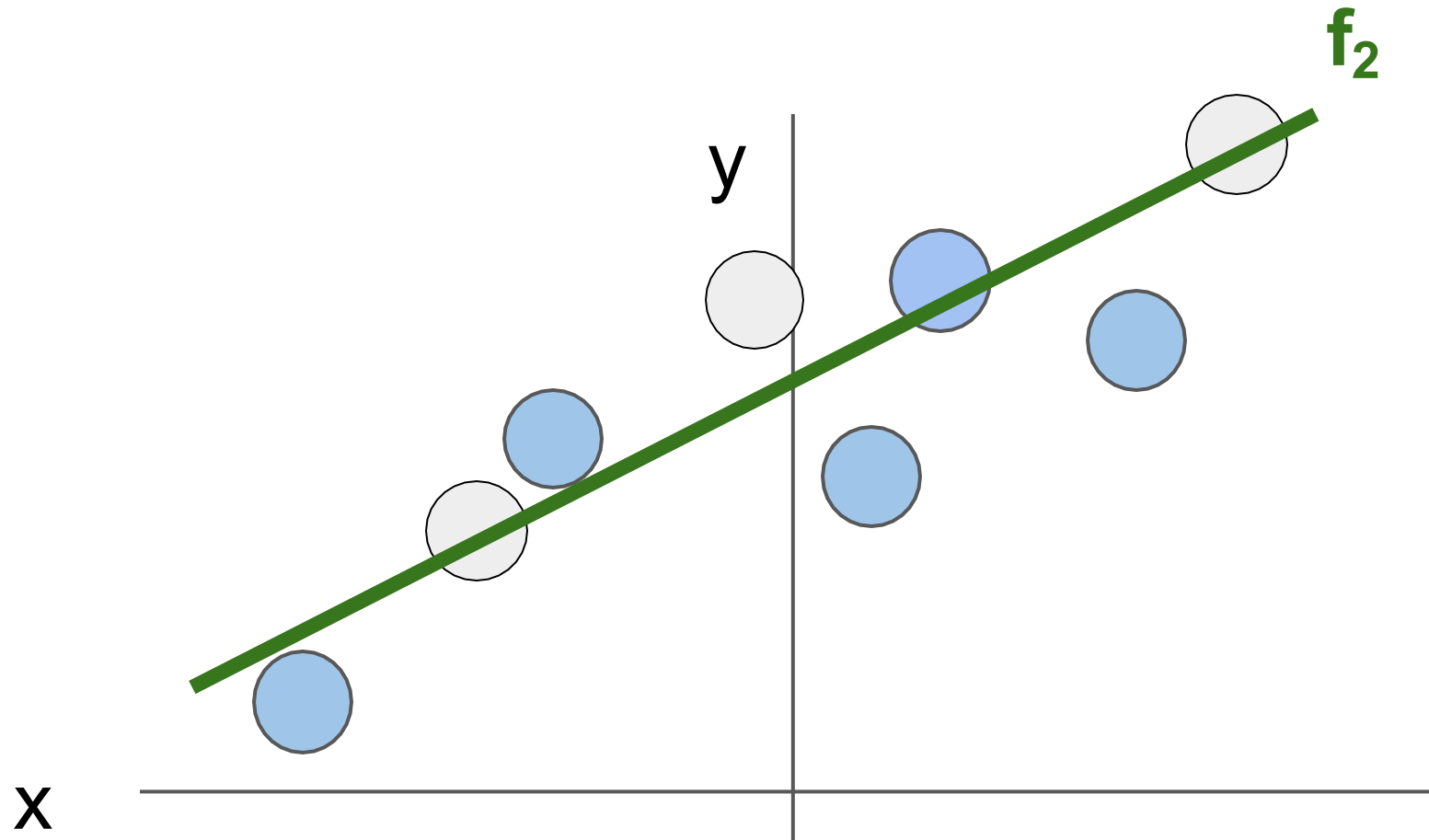
Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

Regularization intuition



Simpler models are more robust



Common terms

- L1 regularization: feature selection

$$R(W) = \sum_k \sum_l |W_{k,l}|$$

- L2 regularization: spread weights

$$R(W) = \sum_k \sum_l W_{k,l}^2$$

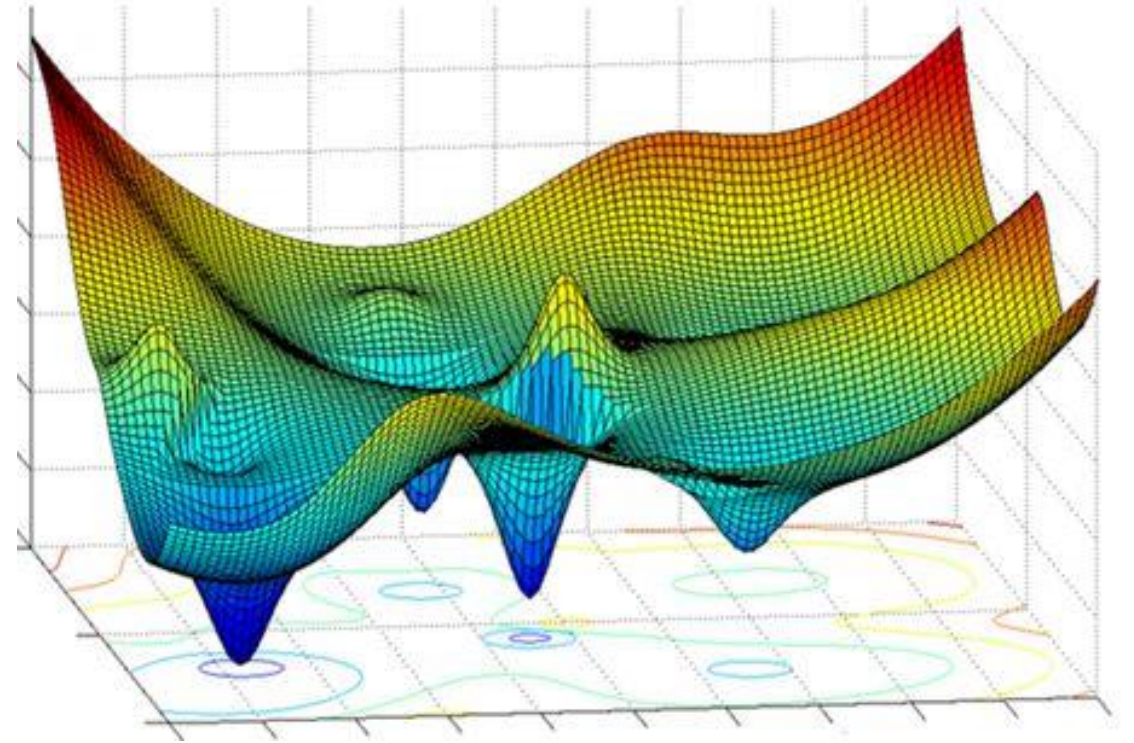
which of w_1 or w_2 L2 prefers?

$$w_1 = [0.5, 0.5, 0.5]$$

$$w_2 = [1, 0, 0]$$

Optimization

- How to find the minimum?

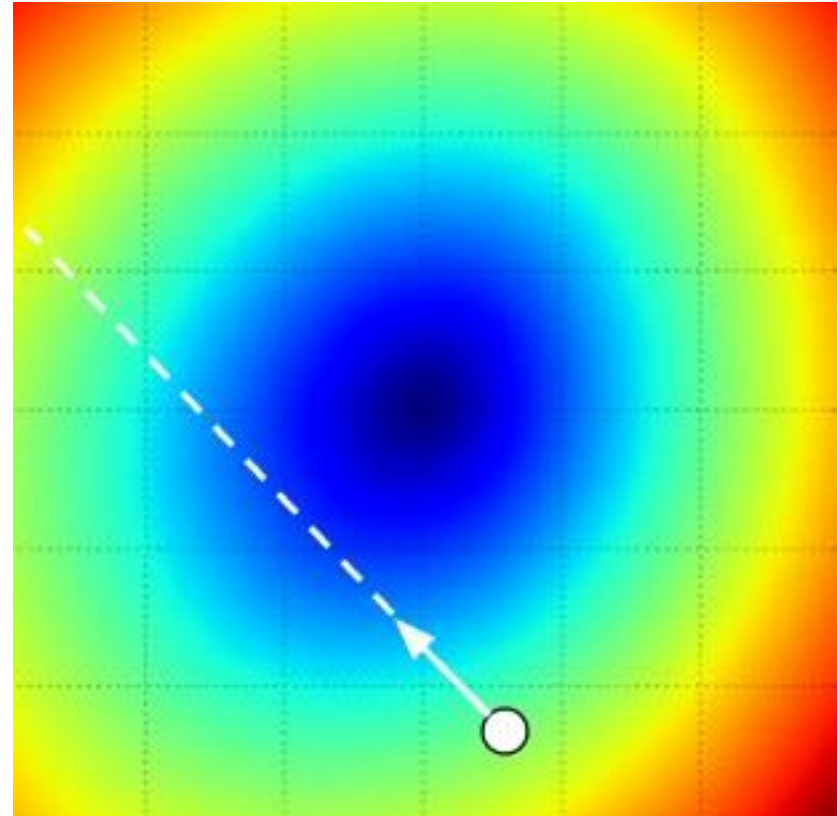


2D loss surface

Optimization with gradient decent

- Update W by following the gradient of the loss function with respect to W

$$w_t = w_{t-1} - \alpha \frac{\partial \mathcal{L}}{\partial w}$$



The Derivative

- In 1D, the derivative is defined as:

$$\frac{df}{dx} = \lim_{h \rightarrow 0} \frac{f(x + h) - f(x)}{h}$$

- In higher dimension, the **gradient** is the vector of (partial derivatives) along each dimension

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

gradient dW:

[?,
?,
?,
?,
?,
?,
?,
?,
?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (first dim):

[0.34 + **0.0001**,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25322

gradient dW:

[?,
?,
?,
?,
?,
?,
?,
?,
?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (first dim):

[0.34 + **0.0001**,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25322

gradient dW:

[-2.5,
?,
?,

$$(1.25322 - 1.25347)/0.0001 = -2.5$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?,
?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (second dim):

[0.34,
-1.11 + **0.0001**,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25353

gradient dW:

[-2.5,
?,
?,
?,
?,
?,
?,
?,
?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (second dim):

[0.34,
-1.11 + **0.0001**,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25353

gradient dW:

[-2.5,
0.6,
?,
?,

$$(1.25353 - 1.25347)/0.0001 = 0.6$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (third dim):

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

gradient dW:

[-2.5,
0.6,
?,
?,
?,
?,
?,
?,
?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

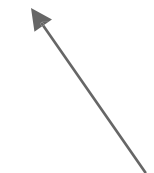
W + h (third dim):

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

gradient dW:

[-2.5,
0.6,
0,
?,
?


$$(1.25347 - 1.25347)/0.0001 = 0$$

$$\frac{df(x)}{dx} = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$$

?,...]

current W:

[0.34,
-1.11,
0.78,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

W + h (third dim):

[0.34,
-1.11,
0.78 + **0.0001**,
0.12,
0.55,
2.81,
-3.1,
-1.5,
0.33,...]

loss 1.25347

gradient dW:

[-2.5,
0.6,
0,
?,
?

Numeric Gradient

- Slow! Need to loop over all dimensions
- Approximate

?,...]

Analytic gradient

- In practice, we can compute the exact derivative of a function through calculus.
- But still, keep in mind that intuitively, the gradient shows how a small change in the weight affects the loss.

gradient dW:

[-2.5,
0.6,
0,
0.2,
0.7,
-0.5,
1.1,
1.3,
-2.1,...]

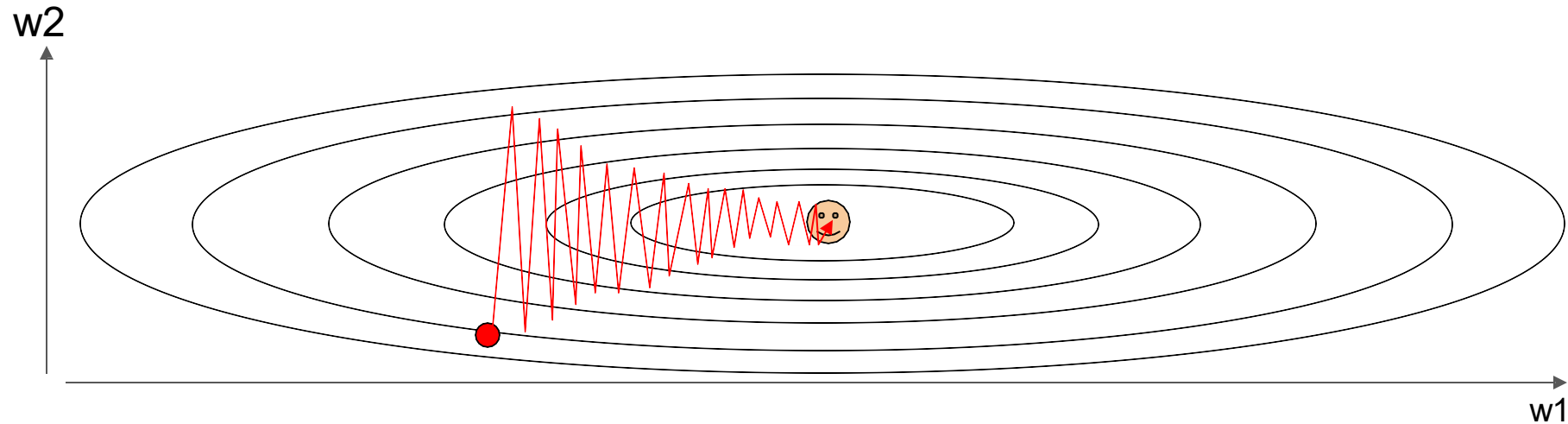
Stochastic Gradient Descent (SGD)

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

- Approximate sum using a minibatch
- Pros:
 - **Efficiency:** Scales well to large datasets and streaming data
 - **Regularization:** Noise in updates helps avoiding local minima
- Hyperparameters:
 - Batch size
 - Data sampling

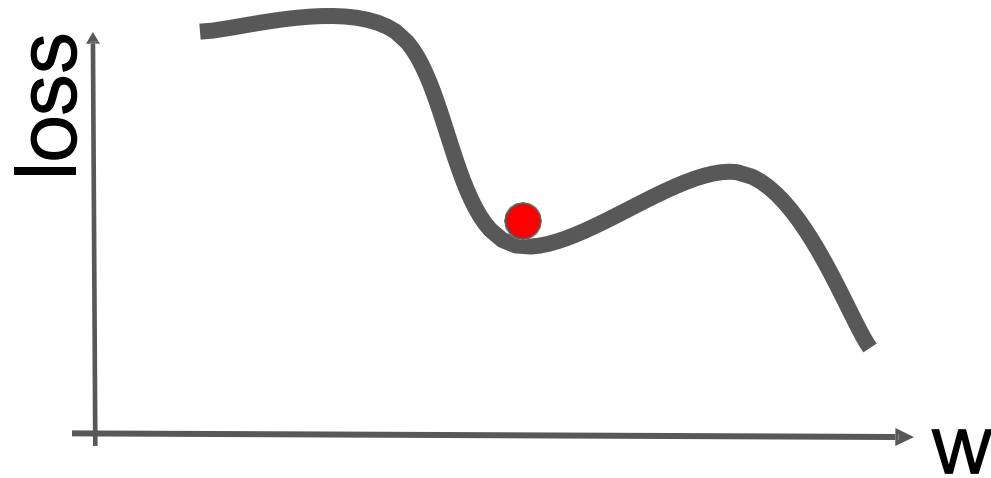
Problem #1

- Very slow progress along shallow dimension, jitter along steep direction

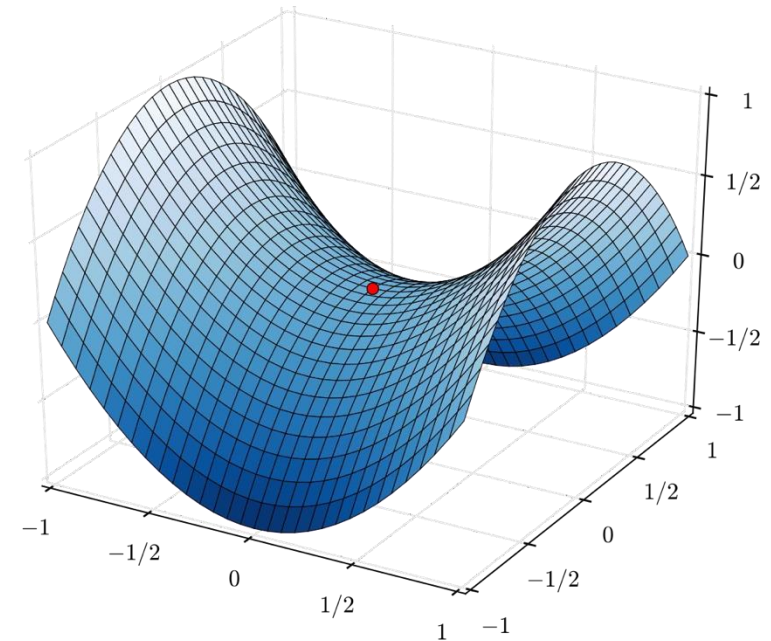


Problem #2

- Local minima or saddle point; Zero gradient



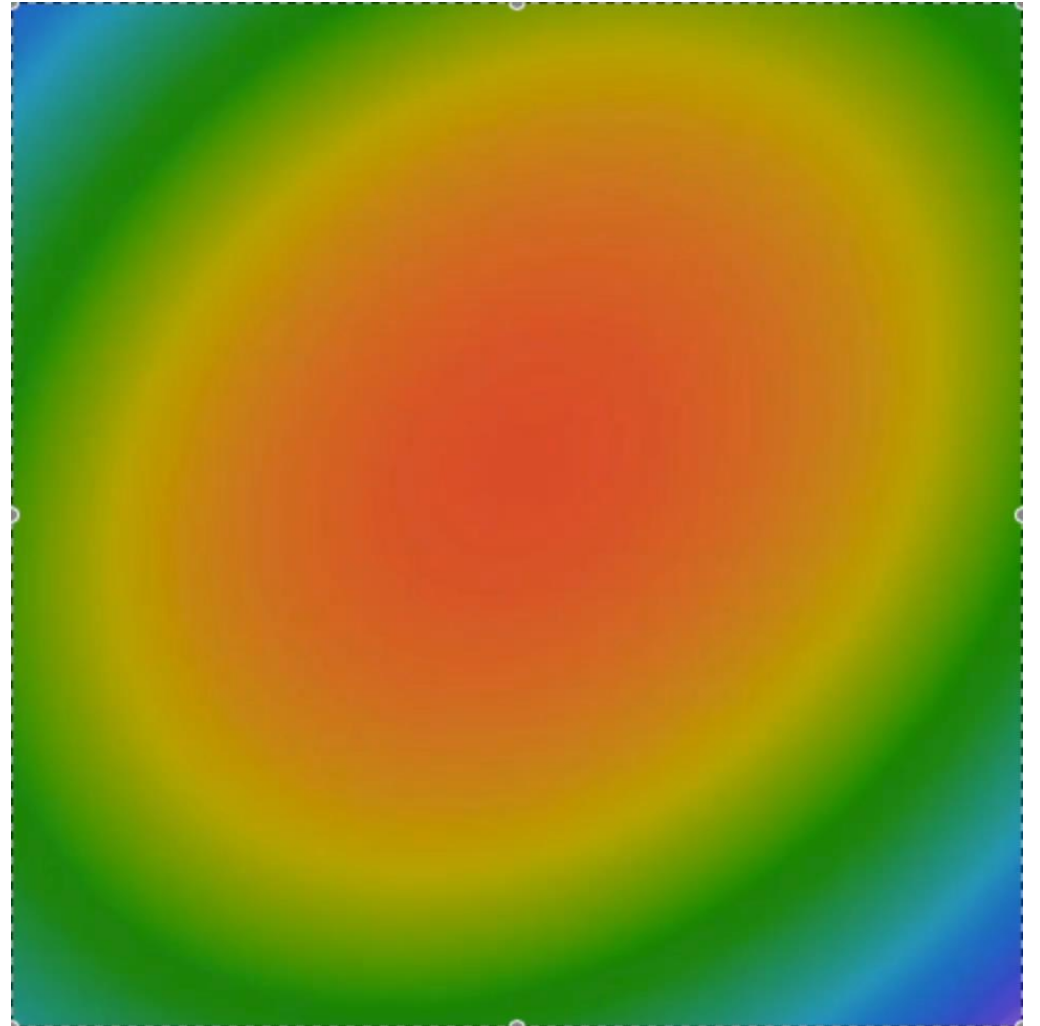
Local minimum



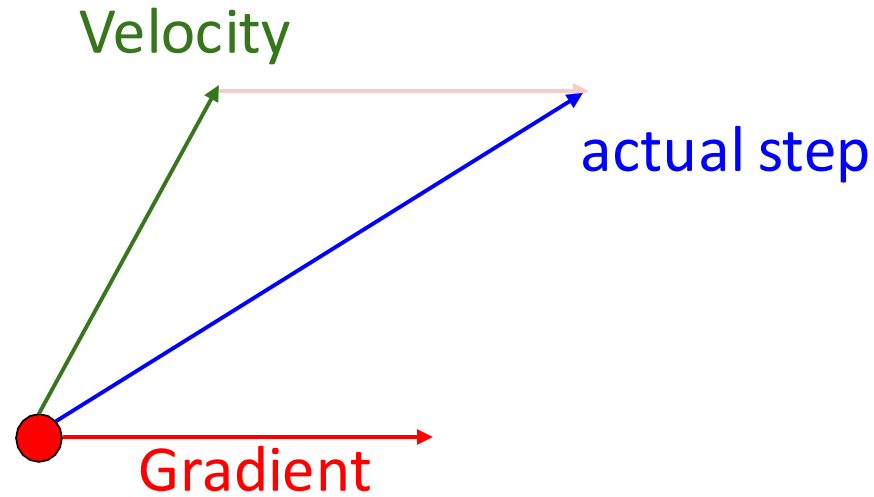
Saddle point

Problem #3

Gradients come from minibatches so they can be noisy.



Momentum update



Combine gradient at current point
with velocity to get step used to
update weights

SGD + Momentum

- Continue direction as the previous iterations
- Build up “velocity” as a running mean of gradients
- Rho gives “momentum”; typically, rho=0.9 or 0.99

$$x_{t+1} = x_t - \alpha \nabla f(x_t)$$

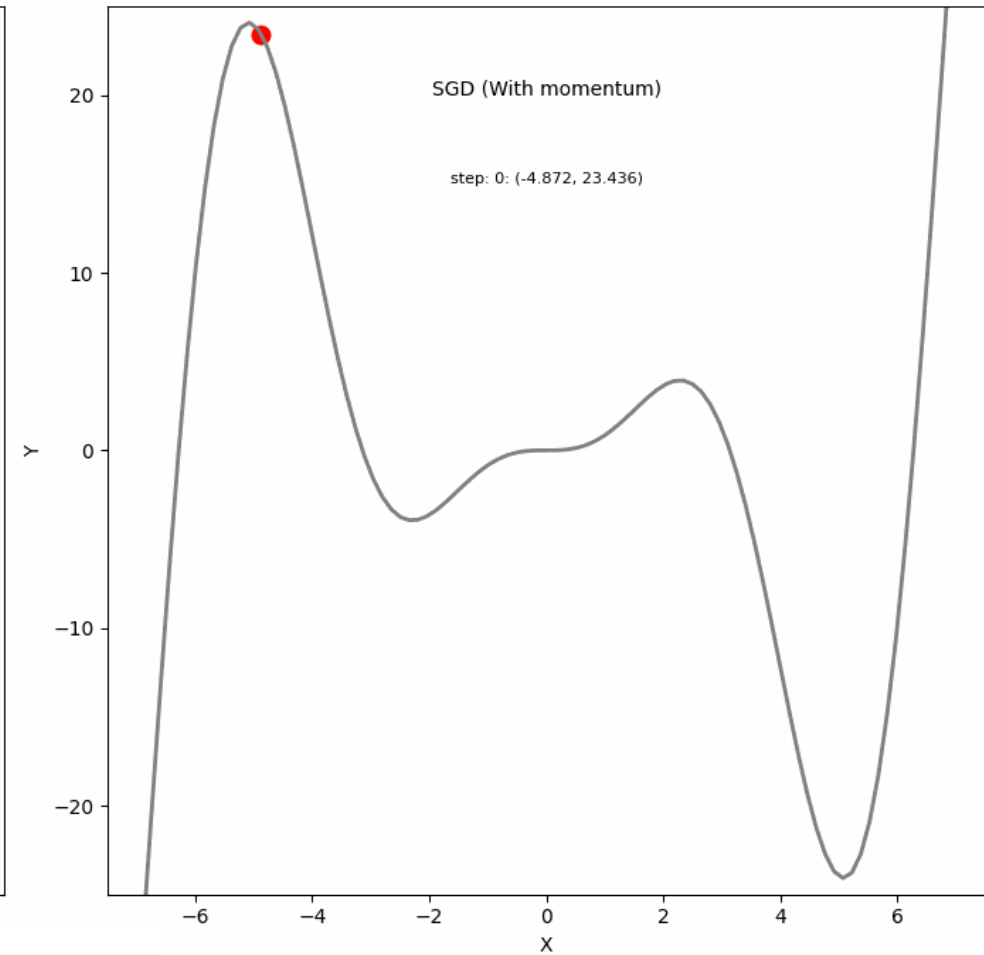
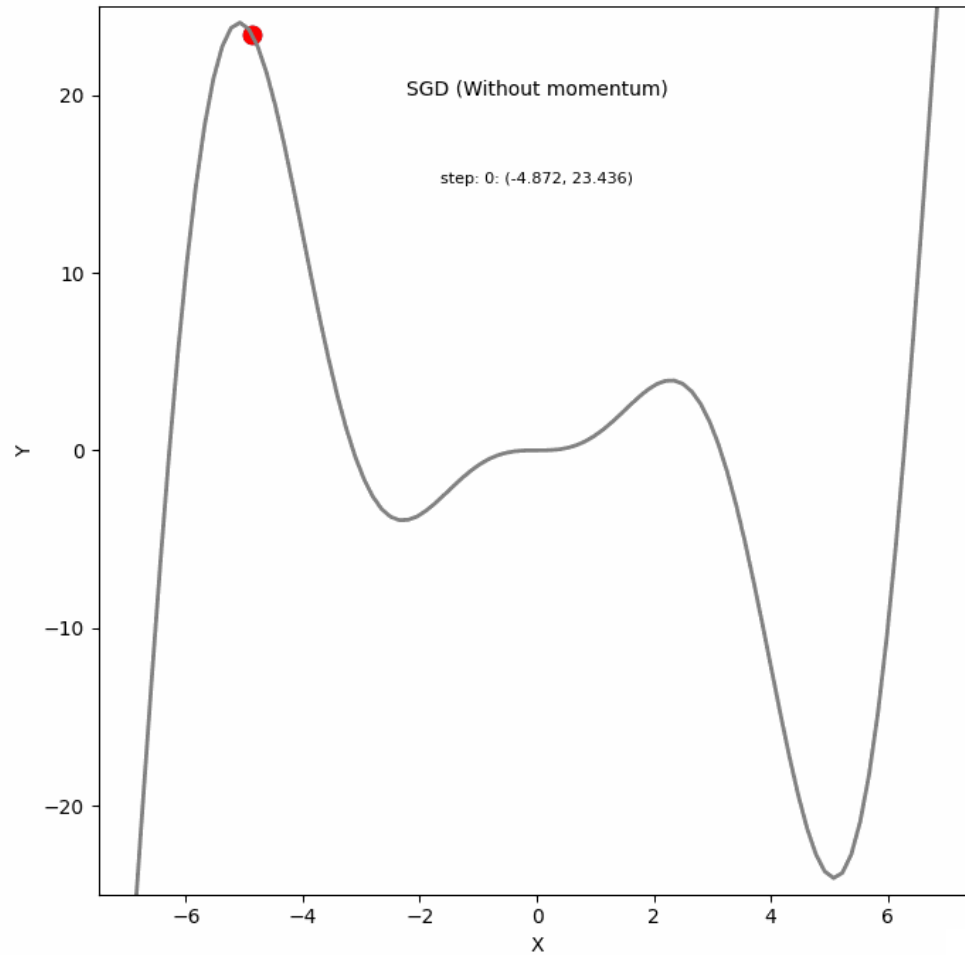
```
for t in range(num_steps):  
    dw = compute_gradient(w)  
    w -= learning_rate * dw
```

$$v_{t+1} = \rho v_t + \nabla f(x_t)$$

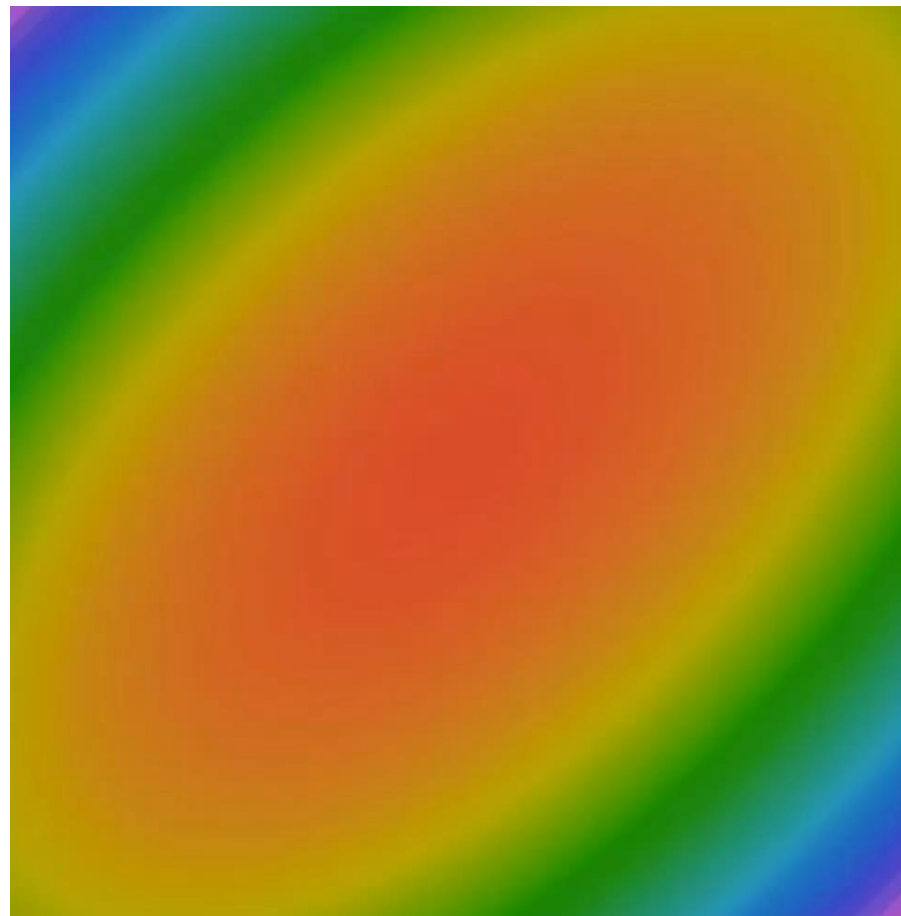
$$x_{t+1} = x_t - \alpha v_{t+1}$$

```
v = 0  
for t in range(num_steps):  
    dw = compute_gradient(w)  
    v = rho * v + dw  
    w -= learning_rate * v
```

SGD + Momentum

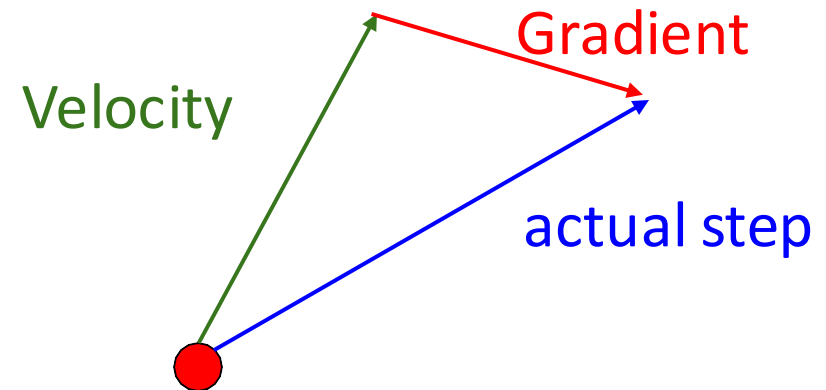


Solves the previous problems



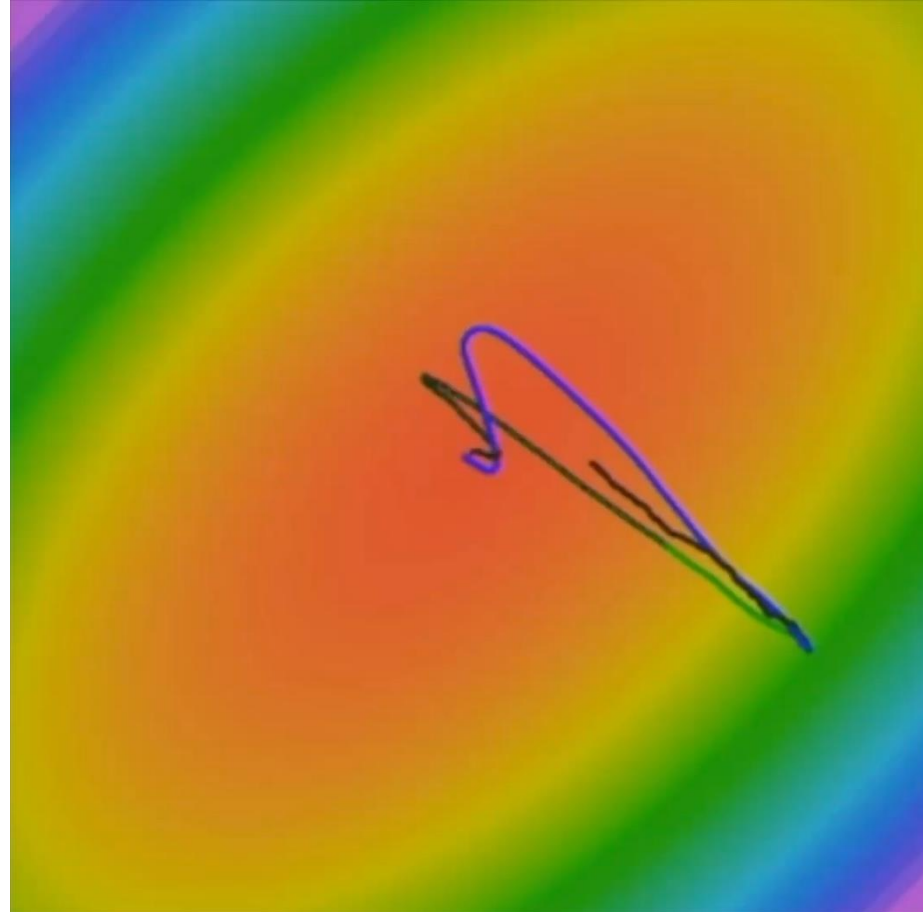
Nesterov Momentum

$$v_{t+1} = \rho v_t - \alpha \nabla f(x_t + \rho v_t)$$
$$x_{t+1} = x_t + v_{t+1}$$

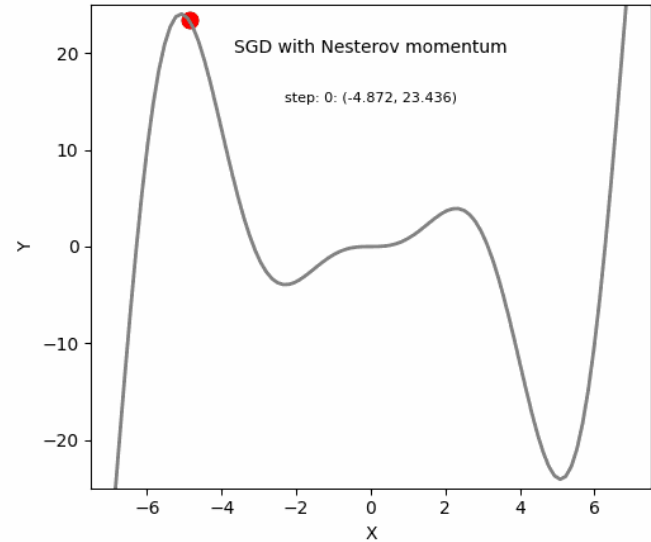
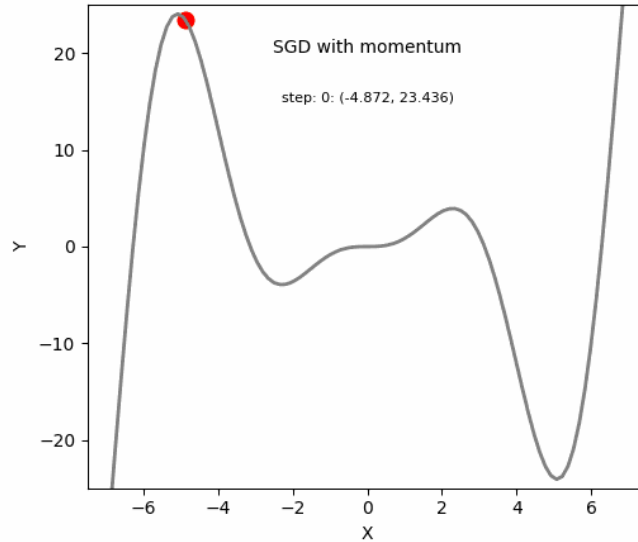
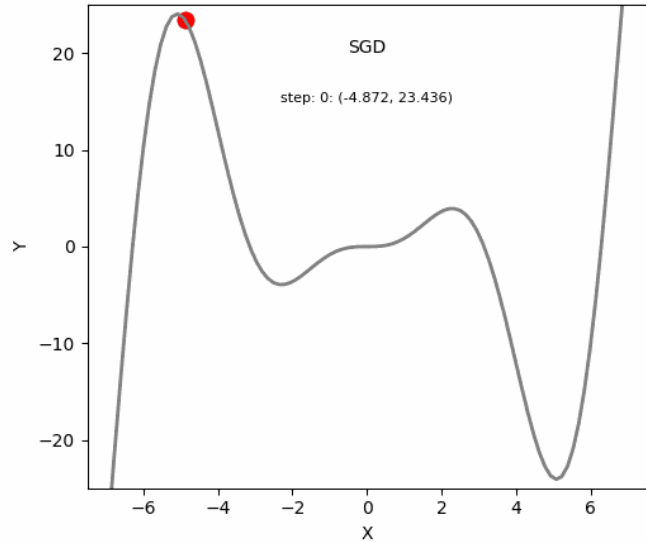
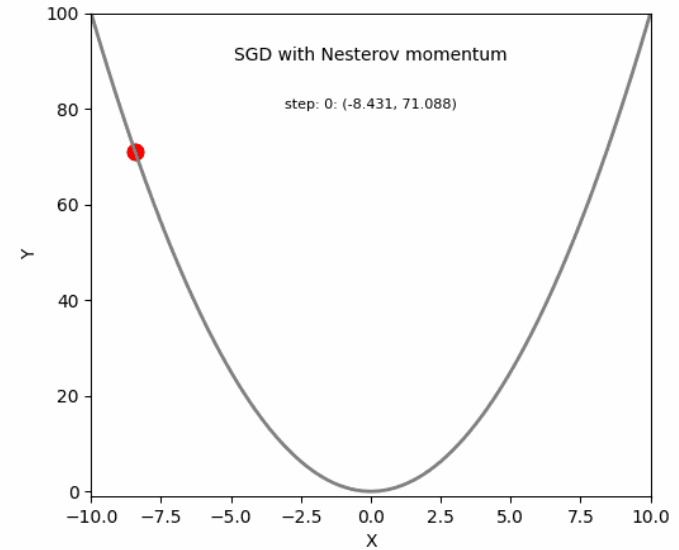
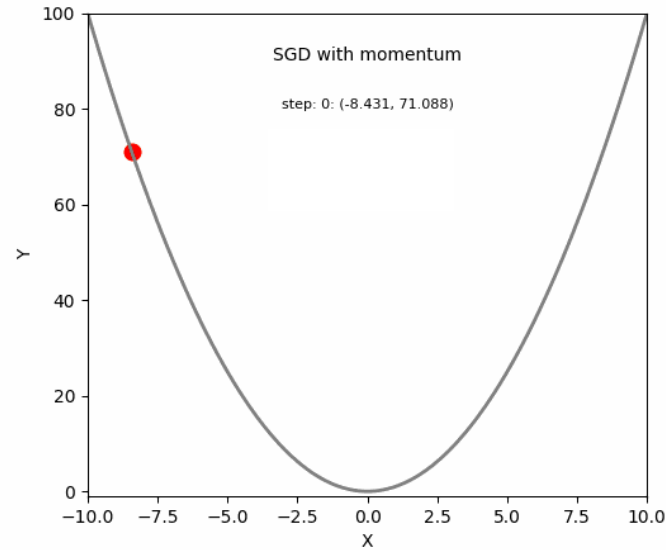
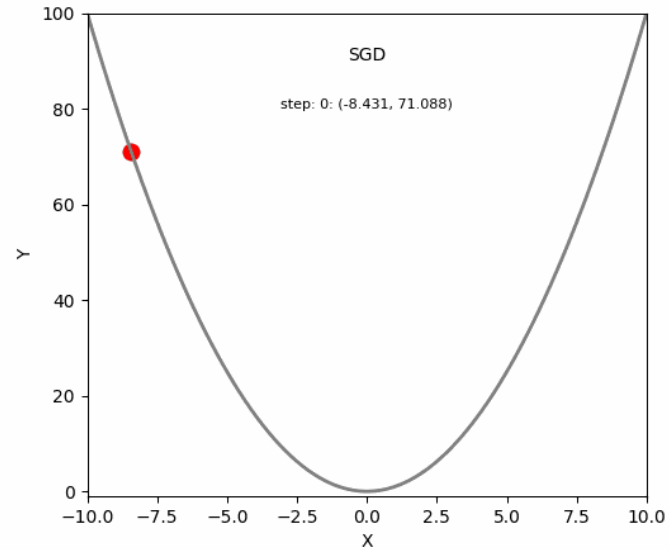


“Look ahead” to the point where updating using velocity would take us; compute gradient there and mix it with velocity to get actual update direction

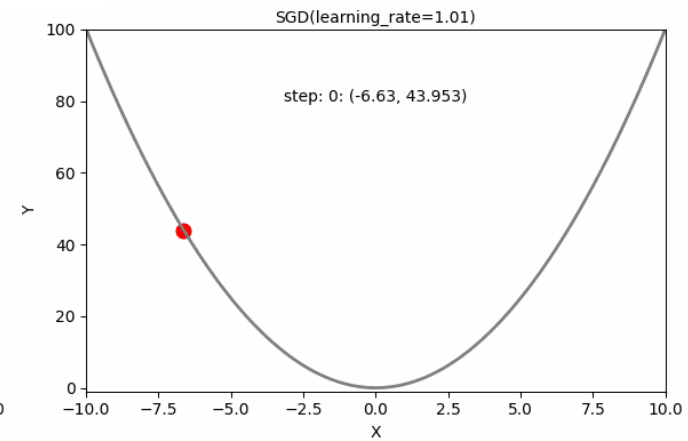
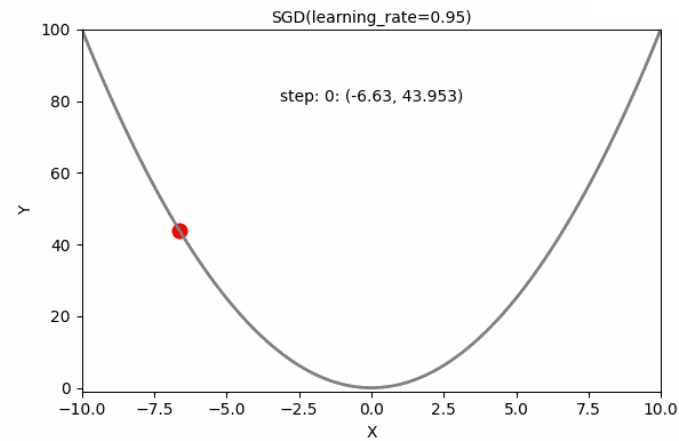
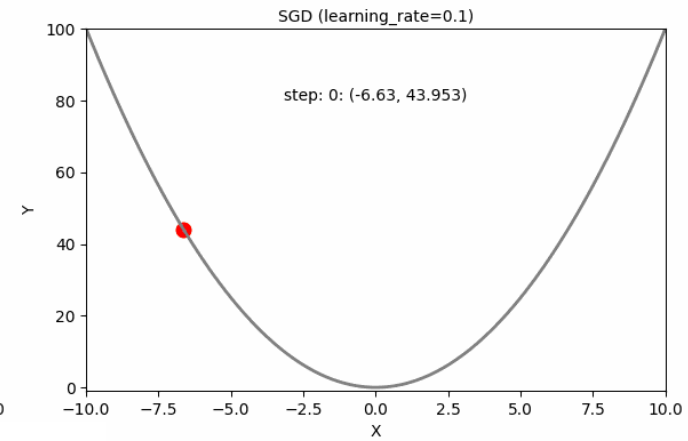
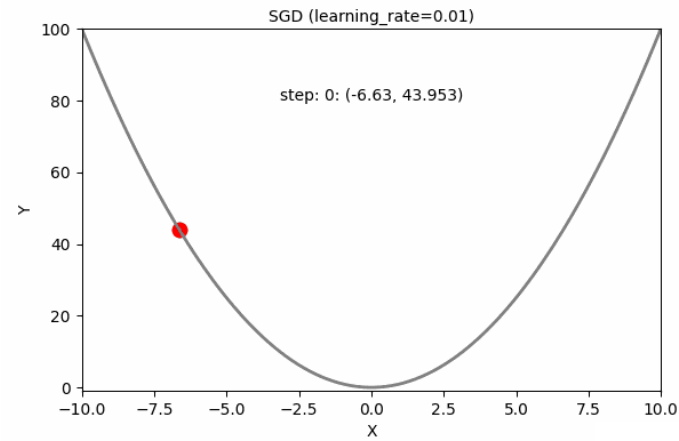
Nesterov Momentum (adds stability)



Nesterov Momentum (adds stability)



Learning rate behavior



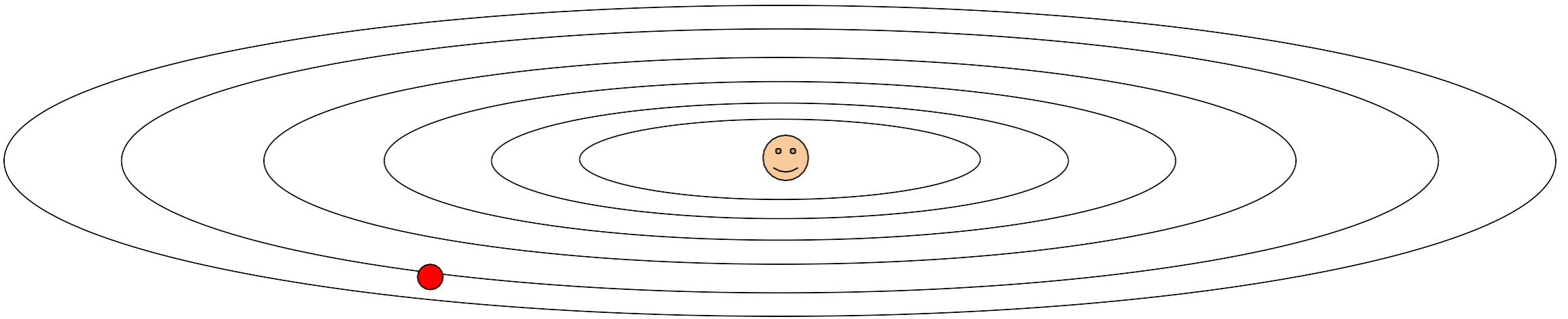
AdaGrad (adaptive learning rates)

- Added element-wise scaling of the gradient based on the historical sum of squares in each dimension

```
grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared += dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```

AdaGrad intuition

```
grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared += dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```



What happens with AdaGrad?

Progress along “steep” directions is damped

Progress along “flat” directions is accelerated

RMSProp: “Leak AdaGrad”

- Prevents the learning rate from shrinking too much

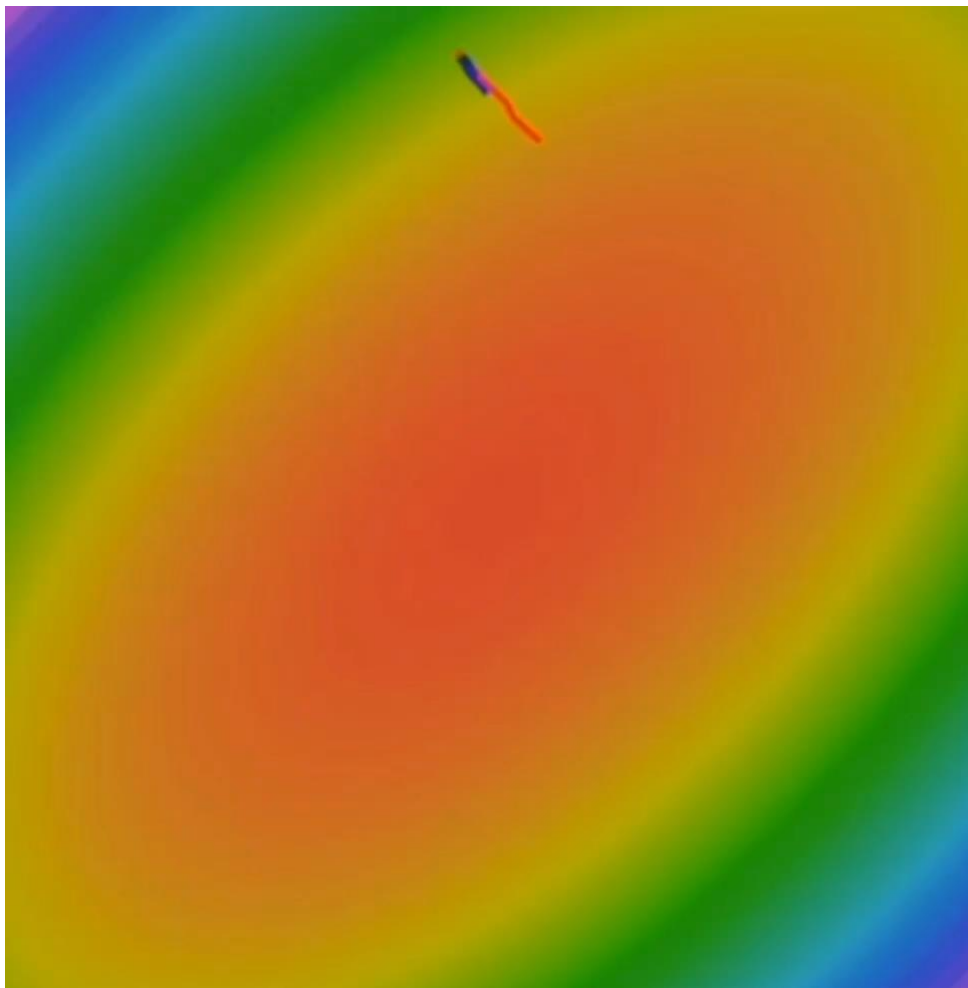
```
grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared += dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```

AdaGrad

```
grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```

RMSProp

RMSProp



— SGD

— SGD+Momentum

— RMSProp

Adam (almost): RMSProp + Momentum

```
moment1 = 0
moment2 = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    moment1 = beta1 * moment1 + (1 - beta1) * dw
    moment2 = beta2 * moment2 + (1 - beta2) * dw * dw
    w -= learning_rate * moment1 / (moment2.sqrt() + 1e-7)
```

Adam

Momentum

```
v = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    v = rho * v + dw
    w -= learning_rate * v
```

SGD+Momentum

Adam (almost): RMSProp + Momentum

```
moment1 = 0
moment2 = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    moment1 = beta1 * moment1 + (1 - beta1) * dw
    moment2 = beta2 * moment2 + (1 - beta2) * dw * dw
    w -= learning_rate * moment1 / (moment2.sqrt() + 1e-7)
```

Adam

Momentum

AdaGrad / RMSProp

```
grad_squared = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    grad_squared = decay_rate * grad_squared + (1 - decay_rate) * dw * dw
    w -= learning_rate * dw / (grad_squared.sqrt() + 1e-7)
```

RMSProp

Very Common in Practice

for input to the CNN; each colored pixel in the image yields a 7D one-hot vector. Following common practice, the network is trained end-to-end using stochastic gradient descent with the Adam optimizer [22]. We anneal the learning rate to 0 using a half cosine schedule without restarts [28].

Bakhtin, van der Maaten, Johnson, Gustafson, and Girshick, NeurIPS 2019

We train all models using Adam [23] with learning rate 10^{-4} and batch size 32 for 1 million iterations; training takes about 3 days on a single Tesla P100. For each mini-batch we first update f , then update D_{img} and D_{obj} .

Johnson, Gupta, and Fei-Fei, CVPR 2018

ganized into three residual blocks. We train for 25 epochs using Adam [27] with learning rate 10^{-4} and 32 images per batch on 8 Tesla V100 GPUs. We set the cubify thresh-

Gkioxari, Malik, and Johnson, ICCV 2019

sampled with each bit drawn uniformly at random. For gradient descent, we use Adam [29] with a learning rate of 10^{-3} and default hyperparameters. All models are trained with batch size 12. Models are trained for 200 epochs, or 400 epochs if being trained on multiple noise layers.

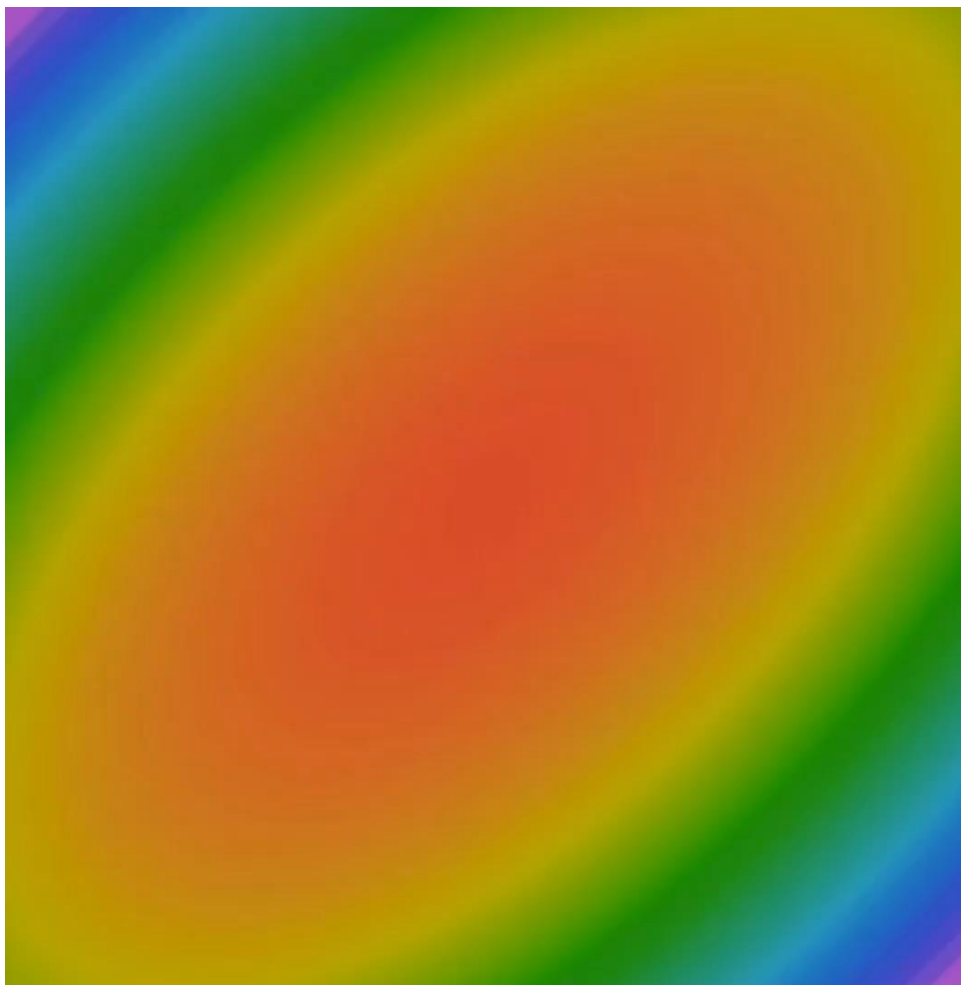
Zhu, Kaplan, Johnson, and Fei-Fei, ECCV 2018

16 dimensional vectors. We iteratively train the Generator and Discriminator with a batch size of 64 for 200 epochs using Adam [22] with an initial learning rate of 0.001.

Gupta, Johnson, et al, CVPR 2018

Adam with $\beta_1 = 0.9$,
 $\beta_2 = 0.999$, and $\text{learning_rate} = 1e-3, 5e-4, 1e-4$
is a great starting point for many models!

Adam



- SGD
- SGD+Momentum
- RMSProp
- Adam

Optimization Algorithm Comparison

Algorithm	Tracks first moments (Momentum)	Tracks second moments (Adaptive learning rates)	Leaky second moments
SGD	X	X	X
SGD+Momentum	✓	X	X
Nesterov	✓	X	X
AdaGrad	X	✓	X
RMSProp	X	✓	✓
Adam	✓	✓	✓

In practice

- Adam is a good default choice in many cases
- SGD+Momentum can outperform Adam but may require more tuning

More recent methods (Muon)

$$\rho(M) = U(aS + bS^3 + cS^5)V^\top$$

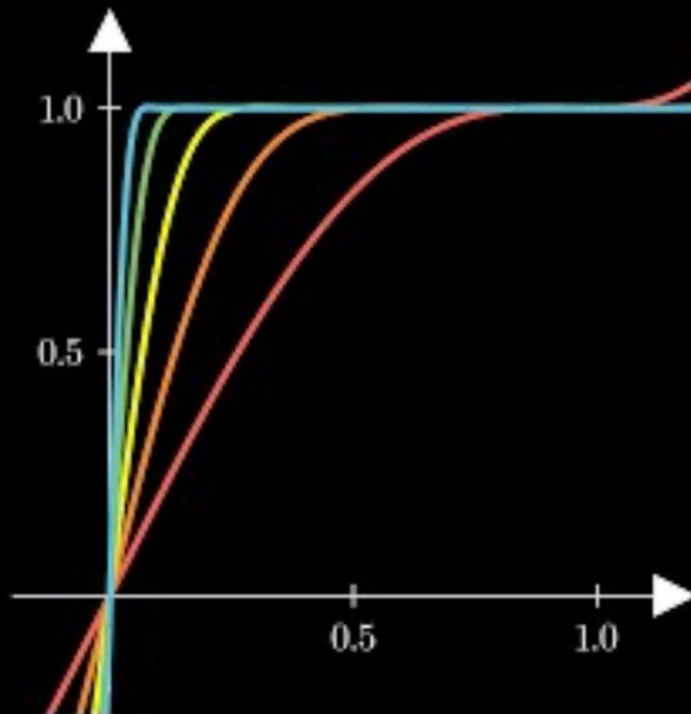
$$y = \rho(x)$$

$$y = \rho(\rho(x))$$

$$y = \rho(\rho(\rho(x)))$$

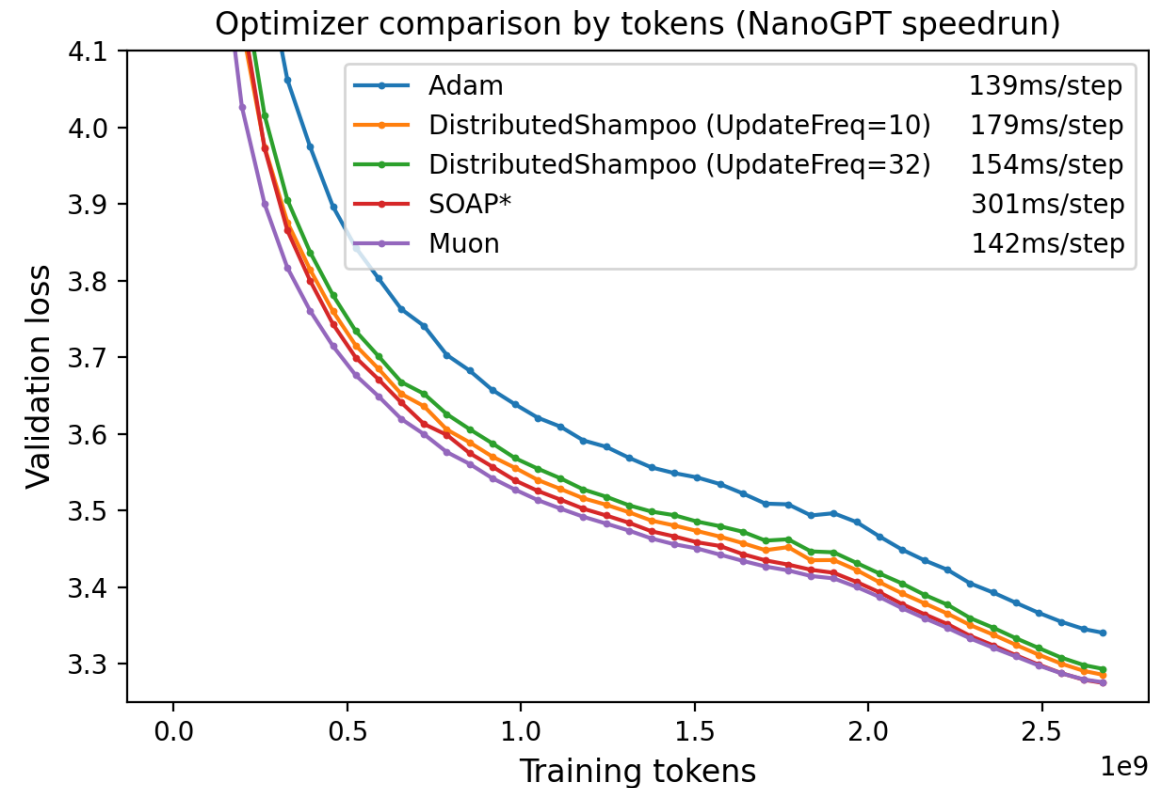
$$y = \rho(\rho(\rho(\rho(x))))$$

$$y = \rho(\rho(\rho(\rho(\rho(x)))))$$



More recent methods (Muon)

- moment_1 and moment_2 require two extra variables per parameter
- Updating each parameter independently is not ideal
 - Vector-based optimization
- Assuming Linear functions can derive much better algorithms



*SOAP is under active development. Future versions will significantly improve the wallclock overhead.

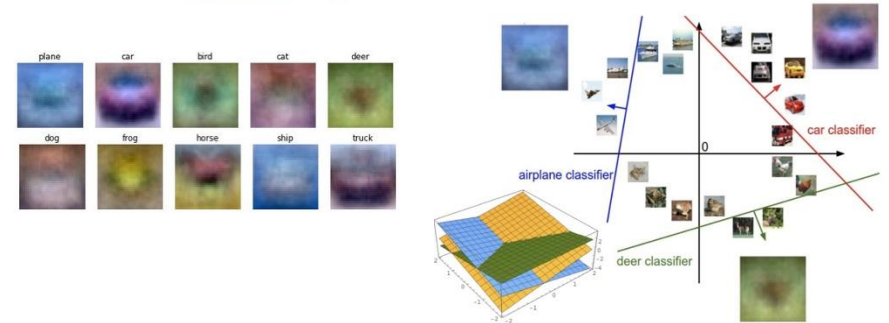
* - <https://www.youtube.com/watch?v=bO5nvE289ec>

* - <https://kellerjordan.github.io/posts/muon/>

Summary

1. Use **Linear Models** for image classification problems
2. Use **Loss Functions** to express preferences over different choices of weights
3. Use **Stochastic Gradient Descent** to minimize our loss functions and train the model

$$s = f(x; W) = Wx$$

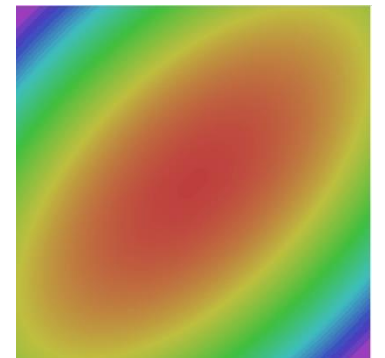


$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right) \quad \text{Softmax} \quad \text{Margin}$$

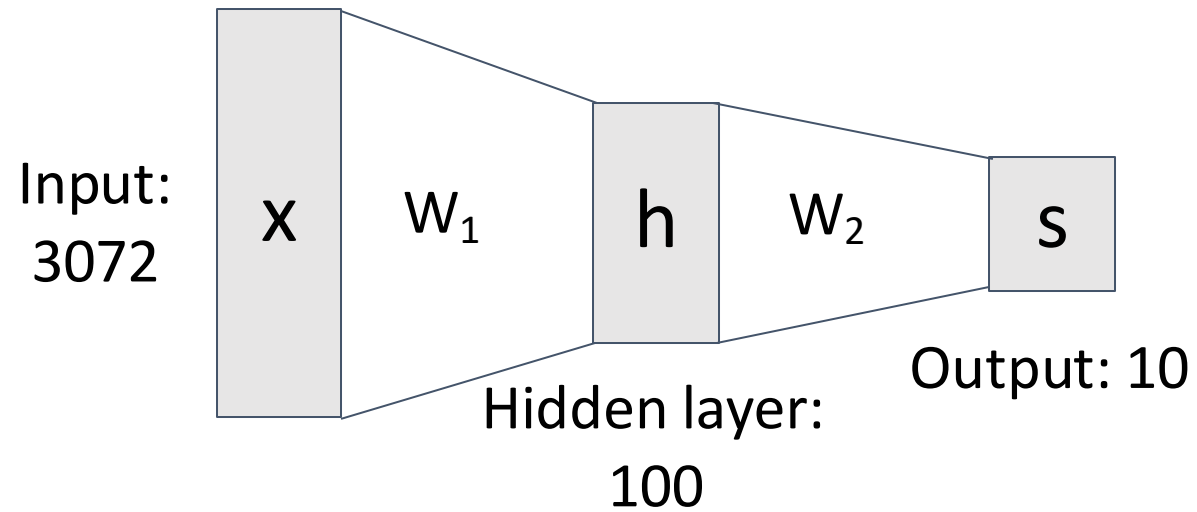
$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$L = \frac{1}{N} \sum_{i=1}^N L_i + R(W)$$

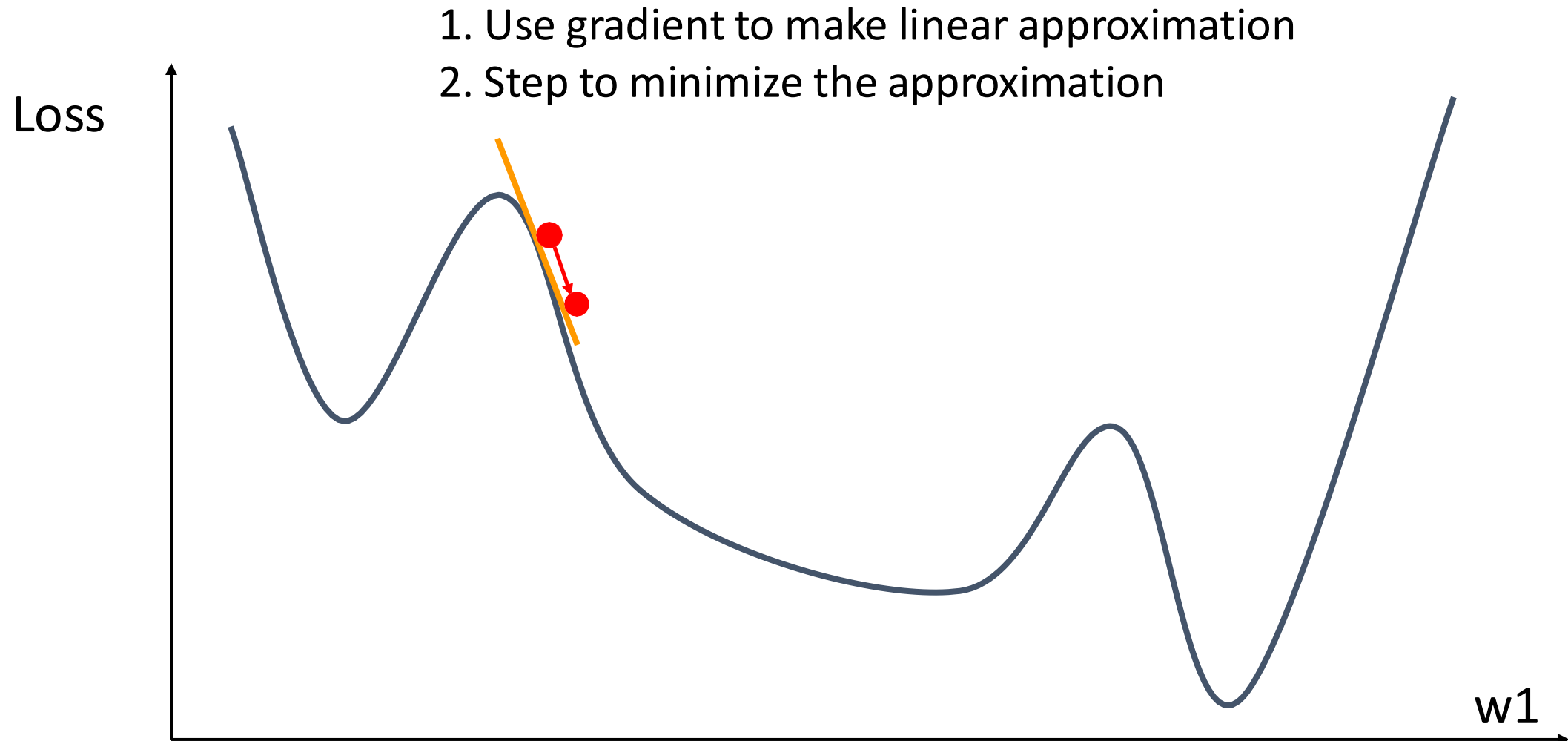
```
v = 0
for t in range(num_steps):
    dw = compute_gradient(w)
    v = rho * v + dw
    w -= learning_rate * v
```



Next: Multi-layer Neural Network

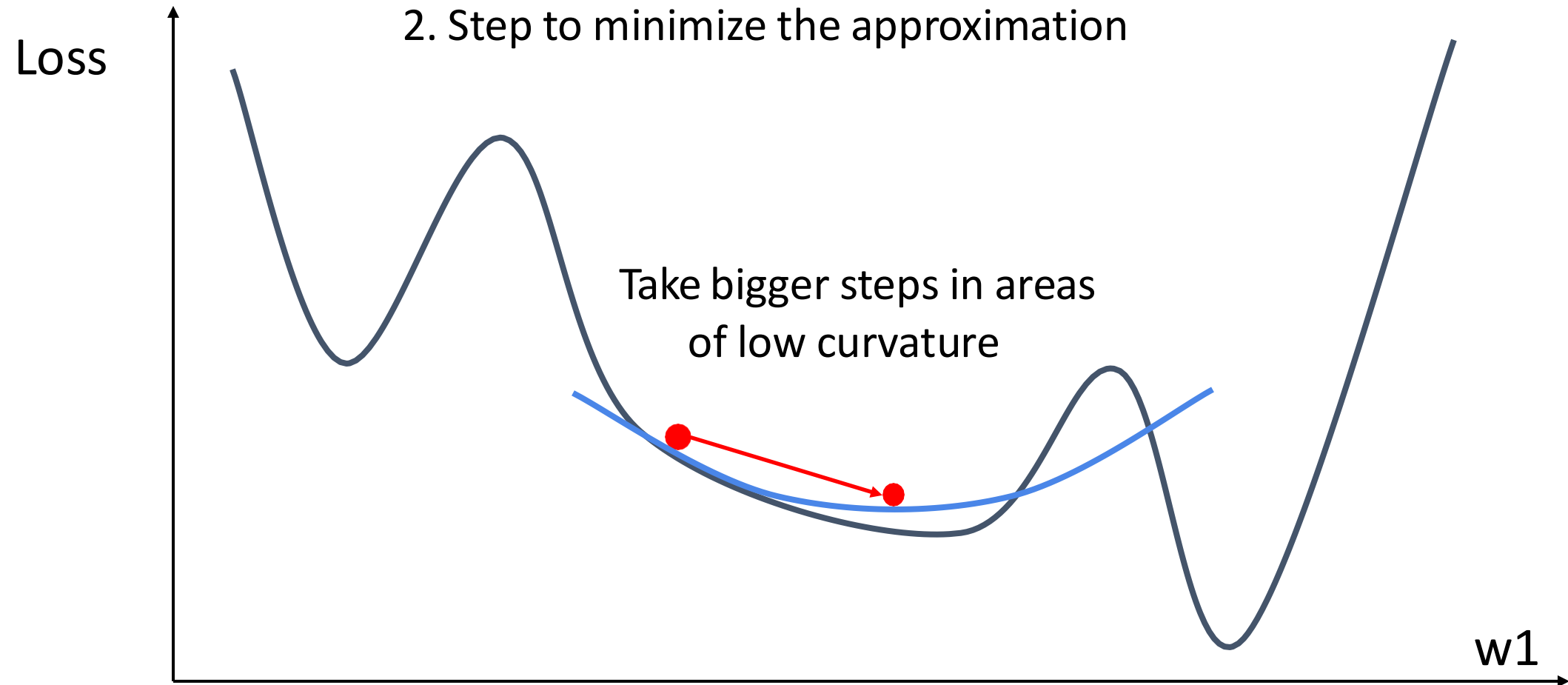


Appendix: First-Order Optimization



Second-Order Optimization

1. Use gradient and Hessian to make quadratic approximation
2. Step to minimize the approximation



Second-Order Optimization

Second-Order Taylor Expansion:

$$L(w) \approx L(w_0) + (w - w_0)^\top \nabla_w L(w_0) + \frac{1}{2} (w - w_0)^\top \mathbf{H}_w L(w_0) (w - w_0)$$

Solving for the critical point we obtain the Newton parameter update:

$$w^* = w_0 - \mathbf{H}_w L(w_0)^{-1} \nabla_w L(w_0)$$

Q: Why is this impractical?

Hessian has $O(N^2)$ elements

Inverting takes $O(N^3)$

N = (Tens or Hundreds of) Millions

Second-Order Optimization

- Quasi-Newton methods (BGFS most popular):
 - instead of inverting the Hessian ($O(n^3)$), approximate inverse Hessian with rank 1 updates over time ($O(n^2)$ each).
- L-BFGS (Limited memory BFGS):
 - Does not form/store the full inverse Hessian.

Second-Order Optimization: L-BFGS

- Usually works very well in full batch, deterministic mode
 - i.e. if you have a single, deterministic $f(x)$ then L-BFGS will probably work very nicely
- Does not transfer very well to mini-batch setting.
 - Gives bad results. Adapting second-order methods to large-scale, stochastic setting is an active area of research.